

ATIVIDADE PRÁTICA EM GRUPO

Test-Driven Development (TDD)

Disciplina: Teste de Software

Modalidade:	Trabalho em Grupo (3 a 5 integrantes)
Entrega:	Repositório GitHub + Apresentação
Linguagem:	Java (preferencialmente) ou outra à escolha
Valor:	10,0 pontos

1. Objetivos da Atividade

Esta atividade tem como objetivo proporcionar aos alunos uma experiência prática e completa com a metodologia Test-Driven Development (TDD), desde a concepção dos requisitos até a demonstração dos benefícios obtidos com a abordagem.

Ao concluir esta atividade, espera-se que os grupos sejam capazes de:

- Compreender e aplicar o ciclo Red-Green-Refactor do TDD
- Elaborar casos de teste antes da implementação do código de produção
- Identificar e descrever os benefícios práticos do TDD no contexto escolhido
- Trabalhar colaborativamente com controle de versão via GitHub
- Comunicar os resultados de forma clara em uma apresentação técnica

2. O que é TDD? Contexto Teórico

O Test-Driven Development é uma prática de desenvolvimento de software proposta por Kent Beck na qual os testes são escritos antes do código de produção. O processo segue um ciclo iterativo de três fases:

O Ciclo Red-Green-Refactor

RED: Escreva um teste que FALHA (a funcionalidade ainda não existe)

GREEN: Escreva o MINIMO de código para o teste passar

REFACTOR: Melhore o código sem quebrar os testes existentes

Diferente da abordagem tradicional (code-first, test-later), o TDD promove:

- Design emergente: o código nasce guiado pelos testes, resultando em interfaces mais limpas
 - Cobertura natural: como os testes são escritos antes, a cobertura é alta por construção
 - Documentação viva: os testes descrevem o comportamento esperado do sistema
 - Confiança para refatorar: a suite de testes serve como rede de segurança
-

3. Descrição da Atividade

3.1 Formação dos Grupos

Os grupos deverão ser formados com 3 a 5 integrantes. Recomenda-se a adoção de *pair programming* rotativo durante a fase de desenvolvimento, onde dois membros trabalham juntos enquanto os demais preparam o próximo ciclo de testes.

3.2 Escolha do Contexto

Cada grupo deverá escolher um contexto de domínio para a aplicação. O contexto deve ser suficientemente rico para comportar ao menos 4 classes e 15 métodos testáveis. Seguem alguns exemplos sugeridos:

Contexto	Exemplos de classes/metodos
Sistema Bancario	Conta, Transferencia, Extrato, CalculadoraJuros
E-commerce	Carrinho, Produto, Cupom, CalculadoraFrete
Sistema de Notas	Aluno, Turma, Media, ClassificadorDesempenho
Estacionamento	Vaga, Veiculo, Ticket, CalculadoraTarifa
Biblioteca	Livro, Emprestimo, Usuario, CalculadoraMulta

Dica: Contexto Próprio

O grupo pode propor um contexto próprio, diferente dos sugeridos acima, desde que seja aprovado pelo professor previamente. Domínios de negócio reais (sistemas que os integrantes já conhecem) tendem a produzir trabalhos mais ricos e com casos de teste mais interessantes.

4. Etapas do Desenvolvimento

Etapa 1 — Levantamento de Requisitos (Semana 1): Entrega em 23/4

O grupo deverá elaborar um documento de requisitos simples contendo:

1. Descrição do contexto escolhido (1 parágrafo)
2. Listagem das classes principais com suas responsabilidades
3. Listagem dos métodos de cada classe com assinatura e comportamento esperado
4. Identificação das regras de negócio que deverão ser validadas pelos testes

Requisitos mínimos obrigatórios:

- Ao menos 4 classes de domínio
- Ao menos 15 métodos testáveis no total
- Ao menos 3 regras de negócio com validação (ex.: saldo não pode ser negativo)
- Ao menos 1 cenário de exceção por classe principal

Etapa 2 — Configuração do Projeto (Semana 1): Entrega em 30/4

Antes de escrever qualquer linha de código de produção, o grupo deverá:

5. Criar o repositório no GitHub (público) com nome significativo
6. Configurar o projeto
7. Adicionar as dependências do framework de teste
8. Criar a estrutura de código
9. Fazer um primeiro *commit* com a estrutura vazia e README.md inicial

Etapa 3 — Implementação com TDD (Semanas 2 e 3): Entrega em 14/5

Esta é a etapa central da atividade. O grupo deverá seguir rigorosamente o ciclo Red-Green-Refactor para cada método implementado. O fluxo obrigatório é:

Fluxo obrigatório por funcionalidade

1. Escreva o teste (ele deve FALHAR - Red)
2. Execute os testes e confirme a falha
3. Implemente o MINIMO de código para o teste passar (Green)
4. Execute os testes e confirme que passam
5. Refatore se necessário, mantendo os testes passando
6. Faça commit após cada ciclo completo

Exemplo de estrutura de teste com JUnit 5 em Java:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
```

```
class ContaBancariaTest {

    private ContaBancaria conta;

    @BeforeEach
    void setUp() {
        conta = new ContaBancaria("001", 1000.0);
    }

    @Test
    @DisplayName("deve realizar saque quando saldo e suficiente")
    void deveSacarQuandoSaldoSuficiente() {
        conta.sacar(200.0);
        assertEquals(800.0, conta.getSaldo(), 0.001);
    }

    @Test
    @DisplayName("deve lancar excecao ao sacar valor maior que o saldo")
    void deveLancarExcecaoAoSacarValorMaiorQueSaldo() {
        assertThrows(SaldoInsuficienteException.class,
            () -> conta.sacar(1500.0));
    }

    @Test
    @DisplayName("nao deve aceitar deposito de valor negativo")
    void naoDeveAceitarDepositoNegativo() {
        assertThrows(IllegalArgumentException.class,
            () -> conta.depositar(-50.0));
    }
}
```

Etapa 4 — Documentação e Análise dos Benefícios (Semana 5): Entrega em 21/5

Após a implementação, o grupo deverá produzir uma análise reflexiva documentando:

10. Evolução do código: como o design emergiu dos testes
11. Bugs encontrados pelos testes antes da integração
12. Comparativo: como seria sem TDD vs. com TDD
13. Lições aprendidas pelo grupo

Etapa 5 — README e Organização do GitHub (Semana 6): Entrega em 28/5

O repositório deve estar bem organizado e conter um README.md completo com:

- Descrição do projeto e contexto escolhido
- Como executar os testes (comando exato)
- Estrutura de pastas comentada
- Resumo dos resultados (cobertura, número de testes)
- Nome de cada integrante

5. Apresentação: De 4 a 25/6

Cada grupo terá 15 minutos para apresentar o trabalho, sendo:

- 5 minutos: contexto escolhido e decisões de design
- 7 minutos: demonstração ao vivo dos testes no repositório
- 3 minutos: análise dos benefícios percebidos e lições aprendidas

Durante a demonstração ao vivo, o grupo deverá mostrar:

14. O histórico de *commits* evidenciando os ciclos Red-Green-Refactor
15. A execução dos testes
16. Ao menos um exemplo completo de ciclo TDD explicado passo a passo

6. Critérios de Avaliação

Critério	Peso	Descrição
Levantamento de Requisitos	1,5	Clareza, completude e adequação dos requisitos
Aplicação do Ciclo TDD	3,0	Evidência nos commits de Red > Green > Refactor
Qualidade dos Testes	2,0	Nomenclatura, cobertura de cenários e assertivas
Código de Produção	1,5	Clareza, organização e aplicação dos princípios SOLID
README e Documentação	1,0	Completude, clareza e organização do repositório
Apresentação	1,0	Clareza, demonstração ao vivo e respostas às perguntas
TOTAL	10,0	

Critério de Desclassificação

Grupos que não evidenciarem o ciclo TDD nos *commits* (ou seja, que escreverem todo o código de produção antes dos testes) terão desconto de 3,0 pontos no critério 'Aplicação do Ciclo TDD', independentemente da qualidade do código entregue.

7. Ferramentas e Recursos

Stack Java Recomendada

- Java 17+ (LTS)
- JUnit 5 (Jupiter) para testes
- Maven ou Gradle como build tool
- Git + GitHub para versionamento

Outras Linguagens Aceitas

Linguagem	Framework de Teste	Cobertura
C# (.NET)	xUnit / NUnit	Coverlet
Python	Pytest	pytest-cov
TypeScript	Jest / Vitest	Istanbul (nyc)
Kotlin	JUnit 5 / Kotest	JaCoCo

8. Dicas para o Sucesso

Dica 1: Nomeie os testes como documentação

O nome do método de teste deve descrever o comportamento esperado.

Use o padrão: `deve[Acao]Quando[Condicao]`

Exemplo: `deveCalcularJurosCompostosQuandoPrazoForMaiorQue30Dias()`

Dica 2: Comece pelos casos mais simples

Inicie pelos testes do caminho feliz (happy path) e depois avance para os casos de borda e exceções. Isso facilita a construção incremental do código de produção.

Dica 3: Um commit por ciclo TDD

Fazer *commits* frequentes (Red, Green, Refactor) é fundamental para demonstrar a aplicação da metodologia. Um histórico rico de *commits* é parte dos critérios de avaliação.

Dica 4: Faça code review entre os integrantes

Antes de fazer *push*, pelo menos um outro integrante deve revisar o teste e o código de produção. Isso melhora a qualidade e garante que o ciclo TDD foi respeitado.

9. Cronograma

Semana	Atividade	Entregavel
Semana 1	Etapas 1 e 2: Requisitos e setup do projeto	Repositório criado + docs/requisitos.md
Semana 2	Etapa 3 (parte 1): TDD das primeiras classes	<i>Commits</i> Red/Green/Refactor no GitHub
Semana 3	Etapa 3 (parte 2) + Etapa 4: TDD completo + análise	Código completo + docs/analise-beneficios.md
Semana 4	Etapa 5 + preparação da apresentação	README finalizado + slides
Semanas seguintes	Apresentações para a turma	Apresentação ao vivo (15 min)

Bom trabalho a todos os grupos!

Red. Green. Refactor. Repeat.